

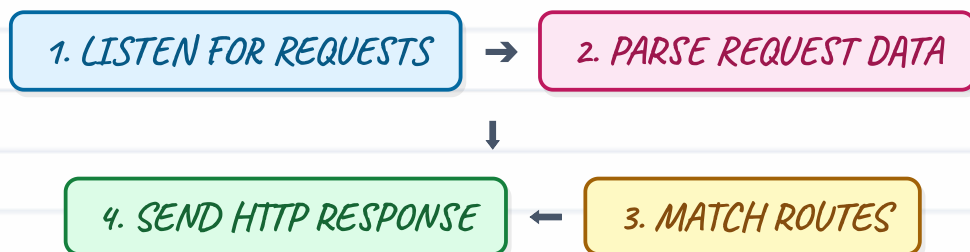
TOPIC 01: LIBRARY VS FRAMEWORK & WHAT IS EXPRESS?

Concept	Definition & Role	Example
Library	A collection of pre-written functions that you call at your convenience to perform specific tasks. You retain control of the application flow.	axios, lodash
Framework	A structured architecture that dictates how your application must be designed. The framework calls your code (Inversion of Control).	Express.js

What is Express.js?

A fast, minimalist, and flexible web application framework for Node.js used for server-side programming and building robust web APIs.

4 Core Responsibilities of Express:



TOPIC 02: SERVER SETUP & APP.USE()

```
const express = require("express"); const app = express(); let port = 3000; //  
Listens to every incoming request regardless of path (Middleware)  
app.use((req, res) => { console.log("Request received!"); }); app.listen(port,  
() => { console.log(`App is listening on port ${port}`); });
```

TOPIC 03: ROUTING, PARAMS & QUERY STRINGS

- **Routing:** The mechanism of mapping client HTTP requests to appropriate server-side handler functions based on URL and HTTP method.
- **Nodemon:** A development utility that automatically monitors and restarts the Node server when directory files change.
- **Wildcard Route (*):** Matches any unmatched route path (used as default 404 handler:
`app.get("*", ...)`).

```
// 1. Path Parameters (req.params): /:username/:id app.get("/:username", (req, res) => { let { username } = req.params; // Extract dynamic URL parameter });  
// 2. Query Strings (req.query): /search?q=apple app.get("/search", (req, res) => { let { q } = req.query; // Extract query parameter });
```

TOPIC 04: EJS (EMBEDDED JAVASCRIPT TEMPLATES)

What is EJS?

EJS is a simple templating language that allows developers to generate **Dynamic HTML** markup using plain JavaScript expressions.

```
app.set("view engine", "ejs"); // Configure EJS template engine
app.get("/", (req, res) => { res.render("home.ejs"); // Renders views/home.ejs });
```



Views Directory Path Rule: To ensure views render reliably regardless of where the server command is executed from, fix the directory path:

```
const path = require("path"); app.set("views", path.join(__dirname, "/views"));
```

TOPIC 05: INTERPOLATION & EJS TAGS

Interpolation: The process of injecting evaluated expressions or variables directly into markup text.

Tag Syntax	Role & Function	Example
<%= %>	Outputs value: Evaluates and inserts value with HTML character escaping.	<%= user.name %>
<% %>	Scriptlet: Executes JavaScript logic (conditionals, loops) without outputting text.	<% if(score >= 50) { %>
<%- %>	Unescaped Output: Renders raw unescaped HTML and includes sub-templates.	<%- include(...) %>
<%# %>	Comment: Internal template comment; excluded from client-side response.	<%# Internal notes %>

TOPIC 06: PASSING DATA & PARTIALS (INCLUDES)

```
// 1. Passing data from route to template app.get("/rolldice", (req, res) => {  
let diceVal = Math.floor(Math.random() * 6) + 1; res.render("rolldice.ejs", {  
num: diceVal }); }); // 2. Modular Sub-templates (Includes): // Inside  
views/home.ejs, reuse common header/nav components: <%-  
include("includes/head.ejs") %>
```

TOPIC 07: GET VS POST REQUEST

Attribute	GET Request	POST Request
Purpose	Retrieve or fetch data from server	Create, submit, or update data on server
Data Transmission	Appended in URL Query String (?key=val)	Sent inside HTTP Request Body (req.body)
Security	Insecure for credentials (exposed in URL history)	Secure: Payload body not exposed in address bar
Payload Size	Restricted by URL length limits	No strict size limits (suitable for large files)

TOPIC 08: PARSING POST REQUEST BODY

Why Middleware is Required:

Express does not parse HTTP request body payloads by default (resulting in undefined). You must register parsing middleware:

```
// 1. Parse URL-encoded HTML Form submissions: app.use(express.urlencoded({
extended: true })); // 2. Parse JSON payload requests:
app.use(express.json()); // Route handler reading parsed body data:
app.post("/register", (req, res) => { let { username, password } = req.body;
res.send(`Welcome ${username}!`); });
```



Key Takeaway: Using `extended: true` leverages the `qs` library allowing rich nested objects to be encoded into the URL-encoded format!

